STREAMLIT DEVELOPER HANDBOOK

End-to-End Master Notes: Core Architecture, Layouts, Plots, CV & Deployment

---

Author: Biswajeet Pradhan (@pradhans369) | **Tech Stack:** Python, Streamlit, Plotly, YOLO, OpenCV
Topic: Full-Stack Modern Python Data Apps & Machine Learning Dashboards

1. Streamlit Architecture & Execution Model

Streamlit turns Python scripts into interactive web apps. Unlike traditional web frameworks (Flask, Django, React) which require routing, HTML templates, and complex state management, Streamlit operates on a **Top-to-Bottom Execution Model**.

The 'Total Amnesia' Execution Rule:

Every time a user interacts with ANY widget (clicks a button, selects a dropdown, moves a slider), **Streamlit reruns the entire Python script from line 1 to the end**. By default, all local variables are re-initialized on every rerun unless explicitly preserved using st.session_state.

> **Core Mental Model**
> Widgets act as return values in Python. For example, btn = st.button('Click') returns True for that single rerun cycle when pressed, and reverts back to False on the next interaction.

Execution Commands & Auto-Runner Pattern:

```
>>> # Terminal command to run your app:
streamlit run app.py

# Python code to allow direct execution with VS Code Play button:
import sys
from streamlit.web import cli as stcli

if __name__ == '__main__':
if st.runtime.exists():
pass
else:
sys.argv = ['streamlit', 'run', sys.argv[0]]
sys.exit(stcli.main())
```

2. Page Configuration & Text Elements

st.set_page_config() **MUST be the very first Streamlit command** called in your script (before any title or widget).

```
>>> st.set_page_config(
page_title='My Dashboard',
page_icon='■',
layout='wide', # 'wide' (full monitor) or 'centered' (narrow)
initial_sidebar_state='expanded'
)
```

Text Formatting Hierarchy:

```
>>> st.title('Main Title (H1)')
st.header('Major Section (H2)')
st.subheader('Subsection Header (H3)')
st.write('General text / Markdown / DataFrames / Dictionaries')
st.caption('Small muted footnote or metadata text')
st.code('import cv2\nimg = cv2.imread("photo.jpg")', language='python')
st.latex(r'''a^2 + b^2 = c^2''')
st.divider() # Draws a sleek horizontal dividing line
```

3. Interactive Input Widgets Reference

Widgets bind Python variables directly to UI controls. When modified, the variable updates and triggers a page rerun.

Widget	Code Signature & Key Arguments	Return Type
`st.button()`	`st.button('Label', key='btn1')`	bool (True on click)
`st.checkbox()`	`st.checkbox('Turn On Camera', value=False)`	bool (Checked state)
`st.selectbox()`	`st.selectbox('Pick', ['A','B','C'], index=None)`	Selected option value
`st.multiselect()`	`st.multiselect('Cols', options, default=['A'])`	list of selected items
`st.slider()`	`st.slider('Age', min_value=0, max_value=100, value=25)`	int / float
`st.file_uploader()`	`st.file_uploader('Upload', type=['csv','mp4'])`	UploadedFile (BytesIO)
`st.camera_input()`	`st.camera_input('Take Live Picture')`	UploadedFile (Snapshot)
`st.text_input()`	`st.text_input('API Key', type='password')`	str (User text)
`st.number_input()`	`st.number_input('Price', min_value=0.0, step=0.5)`	int / float

4. Layouts, Columns, Tabs & Placeholders

1. Multi-Column Layouts:

```
>>> # Proportional columns (left is 1/3, right is 2/3)
col1, col2 = st.columns([1, 2])

with col1:
st.subheader('Inputs')
choice = st.selectbox('Category', ['Tech', 'Finance'])

with col2:
st.subheader('Visualizations')
st.write(f'Showing data for: {choice}')
```

2. Tabs, Expanders & Sidebar:

```
>>> # Sidebar Navigation
with st.sidebar:
st.header('Navigation')
page = st.radio('Go to', ['Overview', 'Model Evaluation', 'Settings'])

# Tabs for clean sub-views
tab1, tab2 = st.tabs(['■ Analytics', '■ Raw Data'])
with tab1:
st.plotly_chart(fig, use_container_width=True)
with tab2:
st.dataframe(df)

# Expandable Accordion
with st.expander('■ Click to view methodology'):
st.write('The model uses YOLOv8 nano trained on COCO.')
```

3. Dynamic Updating Container (st.empty()):

Critical for live video feeds, real-time counters, and animated loops without duplicating elements:

```
>>> # Creates a single placeholder on screen that overwrites itself!
placeholder = st.empty()
for i in range(100):
placeholder.metric('Epoch Progress', f'{i+1}%')
time.sleep(0.05)
```

5. Data Visualization: Plotly, Seaborn & Tableau

Native Plotly Express Integration:

Plotly is the industry standard for interactive charts in Streamlit.

```
>>> import plotly.express as px

# Always calculate clean aggregations in Pandas first
grouped = df.groupby('year', as_index=False)['sales'].sum()

fig = px.bar(
grouped,
x='year',
y='sales',
color='sales',
color_continuous_scale=['green', 'yellow', 'red'],
title='Annual Sales Trend'
)
fig.update_layout(title_x=0.5)

# CRUCIAL: use_container_width=True guarantees mobile/desktop responsiveness
st.plotly_chart(fig, use_container_width=True)
```

Seaborn & Matplotlib (Statistical Plots):

```
>>> import seaborn as sns
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
sns.kdeplot(data=df, x='total_bill', hue='sex', fill=True, ax=ax)
ax.set_title('Bill Distribution by Gender')
st.pyplot(fig)
```

Embedding Tableau Dashboards & Custom HTML:

```
>>> import streamlit.components.v1 as components

# Render full HTML/JavaScript embeds securely
components.html(tableau_embed_code, height=750, scrolling=False)
```

6. State Management (st.session_state) & Caching

1. Preserving Variables Across Reruns (Session State):

Standard Python variables reset on every interaction. st.session_state acts as a persistent dictionary stored in the user's session.

```
>>> # Initialize state key
if 'counter' not in st.session_state:
st.session_state.counter = 0

if st.button('Add +1'):
st.session_state.counter += 1

st.write(f'Current Count: {st.session_state.counter}')
```

2. High-Performance Caching Decorators:

```
>>> # A. For DataFrames, SQL Queries, and Data Cleaning (Serialized to cache)
@st.cache_data
def load_data(filepath):
df = pd.read_csv(filepath)
df['Date'] = pd.to_datetime(df['Date'])
return df

# B. For Heavy Resources: ML Models, Database Connectors, API Clients
@st.cache_resource
def load_yolo():
return YOLO('yolov8n.pt') # Loads weights into RAM ONCE only!
```

7. Computer Vision, OpenCV & YOLO in Streamlit

Integrating live AI models into Streamlit requires careful memory handling, color format conversion, and temporary disk streaming.

Complete Pattern: Real-Time Video Object Detection

```
>>> import streamlit as st
import cv2 as cv
from ultralytics import YOLO
import tempfile

model = YOLO('yolov8n.pt')

rec_vid = st.file_uploader('Upload Video', type=['mp4', 'avi', 'mov'])
if rec_vid is not None:
# 1. Write uploaded bytes to a temp file
tfile = tempfile.NamedTemporaryFile(delete=False, suffix='.mp4')
tfile.write(rec_vid.read())
tfile.close()

if st.button('■ Start AI Detection'):
cap = cv.VideoCapture(tfile.name)
vid_placeholder = st.empty()

while cap.isOpened():
ret, frame = cap.read()
if not ret:
break

# Inference
results = model(frame)
annotated_frame = results[0].plot()

# Convert BGR (OpenCV) to RGB (Streamlit display)
rgb_frame = cv.cvtColor(annotated_frame, cv.COLOR_BGR2RGB)
vid_placeholder.image(rgb_frame, channels='RGB', use_container_width=True)

cap.release()
st.success('■ Processing Finished!')
```

8. Production Deployment & Cloud Secrets

Deploying to **Streamlit Community Cloud** (share.streamlit.io) from GitHub requires strict cloud configuration rules.

1. The requirements.txt File (Cloud Dependencies):

Streamlit Cloud runs headless Linux. Standard opencv-python will crash because it searches for desktop GUI drivers. Always use **opencv-python-headless**:

```
>>> streamlit
ultralytics
opencv-python-headless
numpy
pandas
plotly
pillow
```

2. Secrets Management (Hiding API Keys & Passwords):

```
>>> # Locally in .streamlit/secrets.toml:
# OPENAI_API_KEY = 'sk-proj-xyz...'
# DB_PASSWORD = 'supersecret'

# Inside Python code:
import streamlit as st
api_key = st.secrets['OPENAI_API_KEY']
```

Git Security Rule

Always add .streamlit/secrets.toml to your .gitignore file before committing to GitHub so your private keys are never exposed!